Solution 1. The user should provide a shore of the directed cut. This object must be easy to build when the user is getting his directed cut. This object will be called S in the algorithm.

Algorithm 1 Verify-Directed-Cut(D, B, S)

```
for each a ∈ A(D) do
    v, u ← ends[a]
    if v ∈ S and u ∉ S then
        if a ∉ B then
            return False
        end if
    else if v ∉ S and u ∈ S then
        return False
    else if v ∈ S and u ∈ S then
        if a ∈ B then
            return False
        end if
    else
        if a ∈ B then
            return False
        end if
    end if
end for
return True
```

Solution 2. The only edge on H1min will be the cuts of the vertices that are connected. Besides those, all the other are one of those plus an extra edge added by the fact that you can choose another vertex to be a shore of the new cut. There are two excuses that are when you have a vertex that has no outward edges and loops.

To prove that those are minimal, it is impossible to have just a part of the arcs on this cut. If you change, you will lose the arcs from the vertex that you took from the shore and add new arcs or nothing will change.

All the arcs is one set on H2. Reducing a bit, we can think about all the paths and the minimal sets of arcs that makes no rs paths to exist are one arc of each 'parallel' path or arcs that are from different paths.

$H_1min \subset H_2min$:
first approach is on the notebook
second approach is just say that every element is in H2min. If h1 is a minimal rs cut it means that there is no R such that r $\in$ R $\subset$ V
s with part of those arcs. The rs cut, by definition gets all the outward edges from a set and given that it is impossible to have a rs path between r and s. This is the minimal b because if there was an arc that you can get out of it would be a dead end or a loop.

$H_2min \subset H_1min$:
first approach is to say that every element is in H1min. It is a rs cut because you can have a shore that is all the vertices connected from are to the vertices that the outward edges are on B. If that is not a shore, you can either find a loop or a dead end. And it is minimal because if was not you could discard an edge, but then you can not have a shore, because if it is a rs path you would have at least r with an outward edge in the set.

Solution 3. The following algorithm solves the problem.

Algorithm 2 MinimizeWalk(D, r, s)

$V_{\text{aux}} \leftarrow \emptyset$ ▷ set of vertices of an auxiliary digraph
$A_{\text{aux}} \leftarrow \emptyset$ ▷ set of arcs of an auxiliary digraph
for each $v \in V(D)$ **do**
  $V_{\text{aux}} \leftarrow V_{\text{aux}} \cup v_0 \cup v_0 \cup v_0$
end for
for each $a \in V(A)$ **do**
  $v, u \leftarrow ends[a]$
  $A_{\text{aux}} \leftarrow A_{\text{aux}} \cup v_0u_1 \cup v_1u_2 \cup v_2u_0$
end for
$\pi, d \leftarrow$ Breadth-First-Search$((V_{\text{aux}}, A_{\text{aux}}), v)$
if $d[s_1] < \infty$ **then**
  $p_{\text{aux}} \leftarrow$ Path-From-Branching$((\pi, d), s)$
  p *gets*[] ▷ Will have the rs-walk asked
  for each element in p_{aux} **do**
    ??? Como eu passo de volta pro inicial?
  end for
end if
return NIL

Using the TA's hint, let $D_{\text{aux}}(V_{\text{aux}}, A_{\text{aux}})$ be an auxiliary digraph built from D based on two rules:

- For each $v \in V(D)$, V_{aux} has three vertices v_0, v_1, v_2, so the size of V_{aux} is three times the size of $V(D)$.
- For each arc $vu \in A(D)$, A_{aux} has three arcs v_0u_1, v_1u_2, v_2u_0. Also, A_{aux} is three times the size of $A(D)$.

After building this auxiliary digraph, we run a Breadth-First-Search and verify if there is a path from r_0 to s_1. The idea to do that is that every path that ends on a node labeled with the number 1 is a path where the length $l \equiv 1 \pmod 3$. This is evident because using the auxiliary digraph, every node that you transverse you go from label 0 to 1, 1 to 2, or 2 to 0, meaning that if you start at a vertex with label 0 (r_0), each vertex that you traverse, you are adding a label mod 3, and therefore the length of the path is you find the label mod 3.

Also, if there is a rs walk, where $l \equiv 1 \pmod 3$, this algorithm will find it. First, every possible walk is also possible in the auxiliary digraph given that every node that is reachable from a node on the original graph is also reachable on the auxiliary digraph. This algorithm acount for paths that you go through the same vertex multiple times given that if the rest of the division by three is different for the same thing on the last time you traverse this vertex.

Solution 4. The following algorithm is the solution.

Algorithm 3 Matching-or-Hall-Set $(G, (A, B))$

```
M, K ← Konig's-Algorithm (G, (A, B))
if M saturates A then
    return M                                          ▷ Can I do this?
end if
for each v ∈ A do
    if v ∉ M then
        S ← ∅                                         ▷ S will be the subset described in the question
        π, d ← Breadth-First-Search(G, v)
        for each u ∈ V(G) do
            if (d[u] < ∞) and (u ∈ A) then
                S ← S ∪ u
            end if
        end for
        return S
    end if
end for
```

It is correct because if the connected component always go back to the set A if it goes to the set B, because v and the subsequent are only connected to vertices on the matching, therefore this vertex is connected to a vertex in A.

If the vertex has a neighbor it will go back to the set A because if it was not the case the matching would not be the maximum matching.